

Veille technologique « Les moteurs de jeux vidéo et leurs évolutions techniques»

Tony ripaud

Superviser par Romain Jalabert



plan

- 1 Qu'est-ce qu'un moteur de jeu vidéo ?
- 2 Comment les moteurs ont-ils évolué ?
- 3 Où en sommes-nous aujourd'hui ?
- 4 Le multijoueur et les serveurs
- 5 Les moteurs propriétaires
- 6 conclusion
- 7 sources

```
image.load('graphics/Sky.png').convert()  
image.load('graphics/ground.png').convert()  
render('My game', False, 'Black')
```

```
image.load('graphics/snail/snail1.png').convert_alpha()
```

```
image.load('graphics/player/player_walk_1.png').convert_alpha()
```

```
ent.get():  
pygame.QUIT
```

```
face, (0,0))  
face, (0,300)  
face, (300,50)  
snail_x_pos = 800  
face, (snail_x_pos, 250))  
lef, (80,200))
```

```
Python 3.9.0)
```

unity. <https://www.pygame.org/contribute.html>



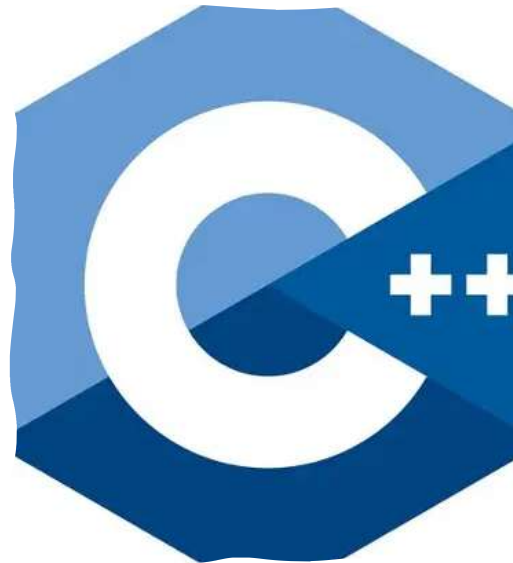
Qu'est-ce qu'un moteur de jeu vidéo ?

- Mutualisation des outils
- Ne pas recréer tout les composants du code
- Eviter les doublons de code
- Centraliser les apelles d'api
- Eviter de recréer l'infrastructure

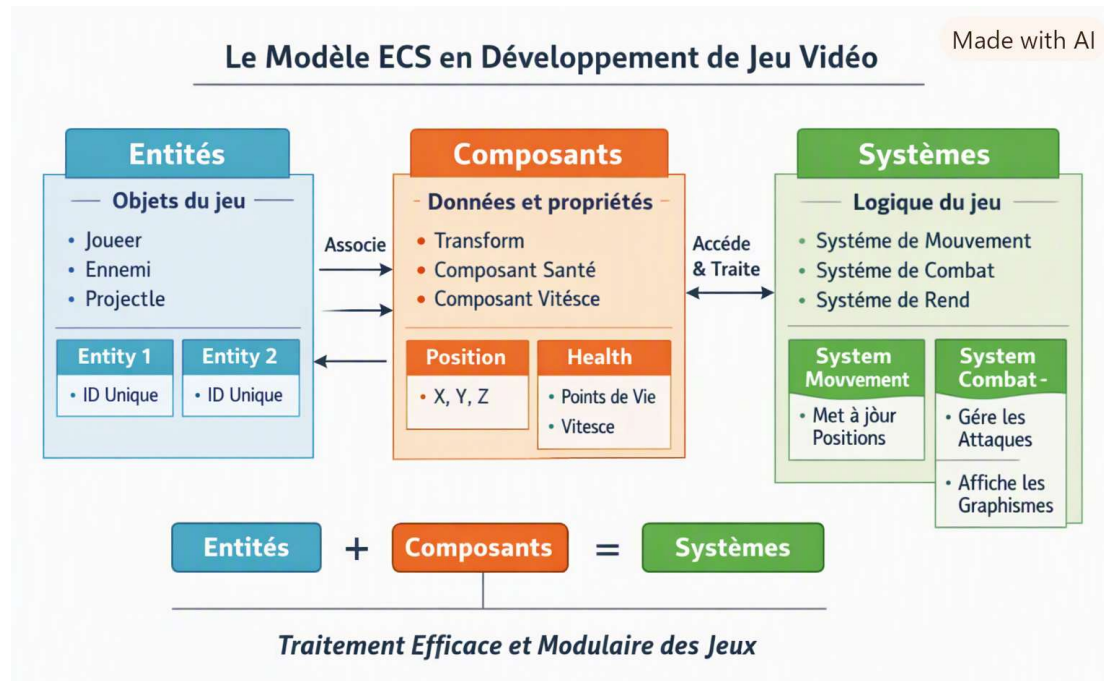


Comment les moteurs ont-ils évolué ?

- Guerres des bits
- 2D->3D
- Les langages (C++, C#, lua, gdscript)
- Spécialisation des moteurs



Où en sommes-nous aujourd'hui ?



- Le ray tracing
- L'ECS (entry component système)
- Languages complémentaires (C# + C++)

L'ECS, c'est au jeu vidéo ce que React est au web — une manière de structurer le code pour séparer la logique, les données et la présentation, tout en favorisant la réutilisation et la performance.

Le multijoueur et les serveurs

```
Csharp ^ Copier

[ServerRpc]
public void ShootServerRpc(Vector3 direction)
{
    // Logique exécutée sur le serveur
    SpawnProjectile(direction);
}

Et côté client :
```

```
Csharp ^ Copier

void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        ShootServerRpc(transform.forward);
    }
}
```

- L'utilisation d'api pour faciliter la partie online
- **RPC**, *Remote Procedure Call*.
- Serveur autoritaire (filtre script sur l'action)

Les moteurs propriétaires

- Re engine-(Capcom)
- Frostbite-(EA)
- RAGE Engine-(rockstar)
- IW Engine — (Infinity Ward / Activision)

The logo for the RAGE engine, featuring the word "RAGE" in a bold, orange, sans-serif font. A yellow star is positioned below the letter "A". The background is a dark, textured surface resembling water or a rocky terrain.The Frostbite logo, featuring a white, stylized snowflake or crystalline structure above the word "FROSTBITE" in a white, sans-serif font. The background is black.The RE Engine logo, featuring a white, stylized hand or claw shape above the words "RE ENGINE" in a white, sans-serif font. The background is black.

conclusion

Les moteurs de jeu permettent aujourd'hui de faciliter coté des des tâches et avec les langages de C++ et C# qui se complémente et utilise les ressource machine il est plus simple d'optimiser le code

```
1 import pygame
2 from pygame.sprite import SpriteCollide
3 import pytmx
4 import pycscroll
5 import random
6 from player import Player
7 from zombie1 import Zombie
8
9 class Game:
10
11     def __init__(self):
12         pygame.font.init()
13         self.health=3
14         self.screen = pygame.display.set_mode((800, 600))
15
16         # Démarrage
17         self.running = True
18         self.map = "world"
19         self.tour=1
20         self.tele=3
21         # Affichage de la fenêtre
22
23         pygame.display.set_caption("ZombieLand")
24
25         # Charger la carte classique
26         tmx_data = pytmx.util_pygame.load_pygame("carte.tmx")
27         map_data = pycscroll.data.TiledMapData(tmx_data)
28         map_layer = pycscroll.orthographic.BufferedRenderer(map_data, self.screen.get_size())
29         map_layer.zoom = 1.25
30
31         myfont = pygame.font.SysFont("monospace", 74)
32         self.score_display = myfont.render("health :",self.health, 1, (0,0,0))
33         self.nombre_zombie=3
34         # Générer le joueur
35
36         player_position = tmx_data.get_object_by_name("player")
37         self.player = Player(player_position.x, player_position.y)
38
```


sources

- <https://docs.unity.com/en-us/engine/6000.7/manual/packages-list/feature-sets/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>
- <https://learn.microsoft.com/en-us/cpp/cpp/>
- <https://www.gamedeveloper.com/>
- <https://godotengine.org/blog/news>

